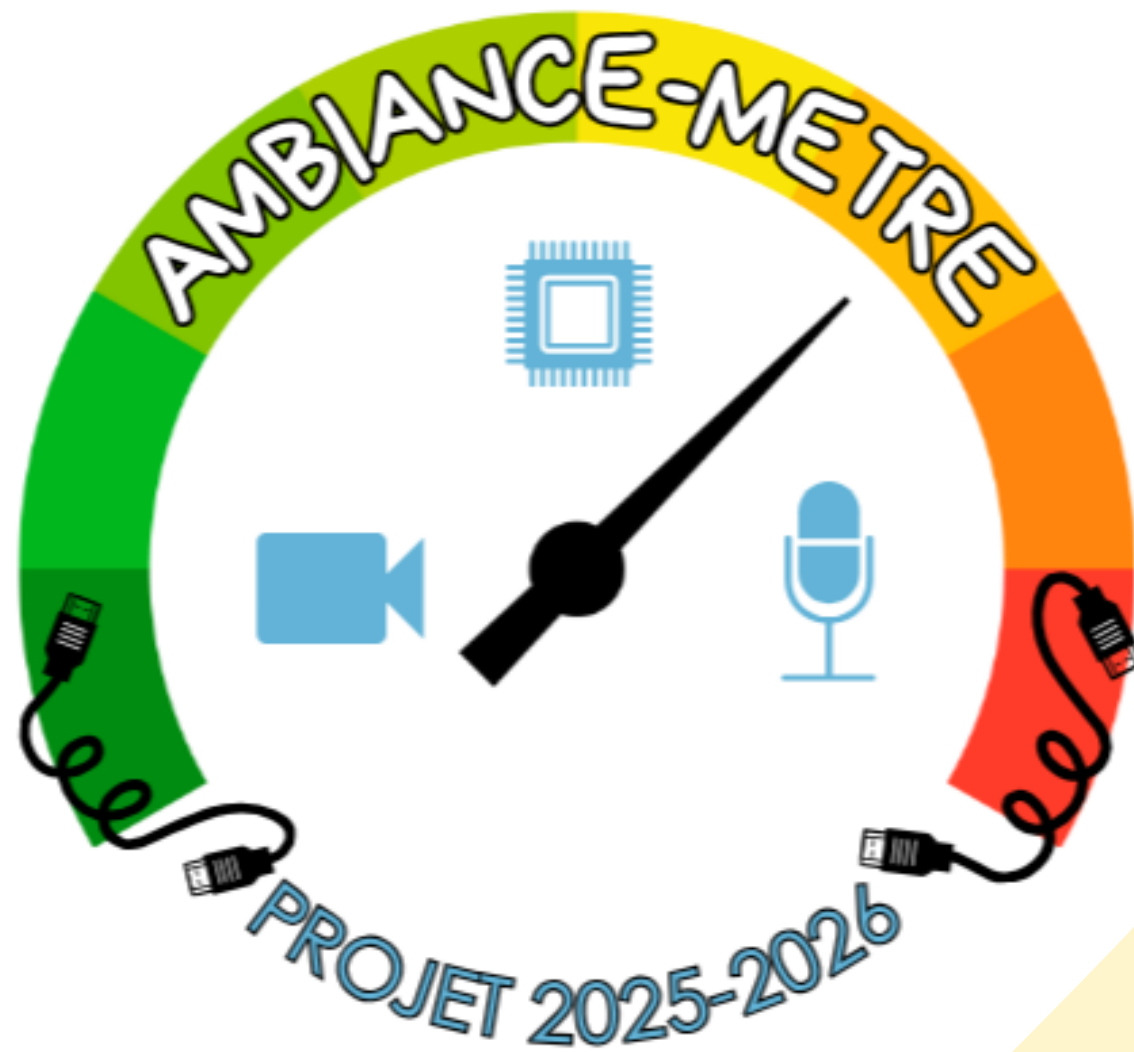


Projet TransDisciplinaire : L'Ambiance-mètre



Sommaire

1

**Explication de
notre projet**

2

Gestion de projet

3

Etapes réalisées

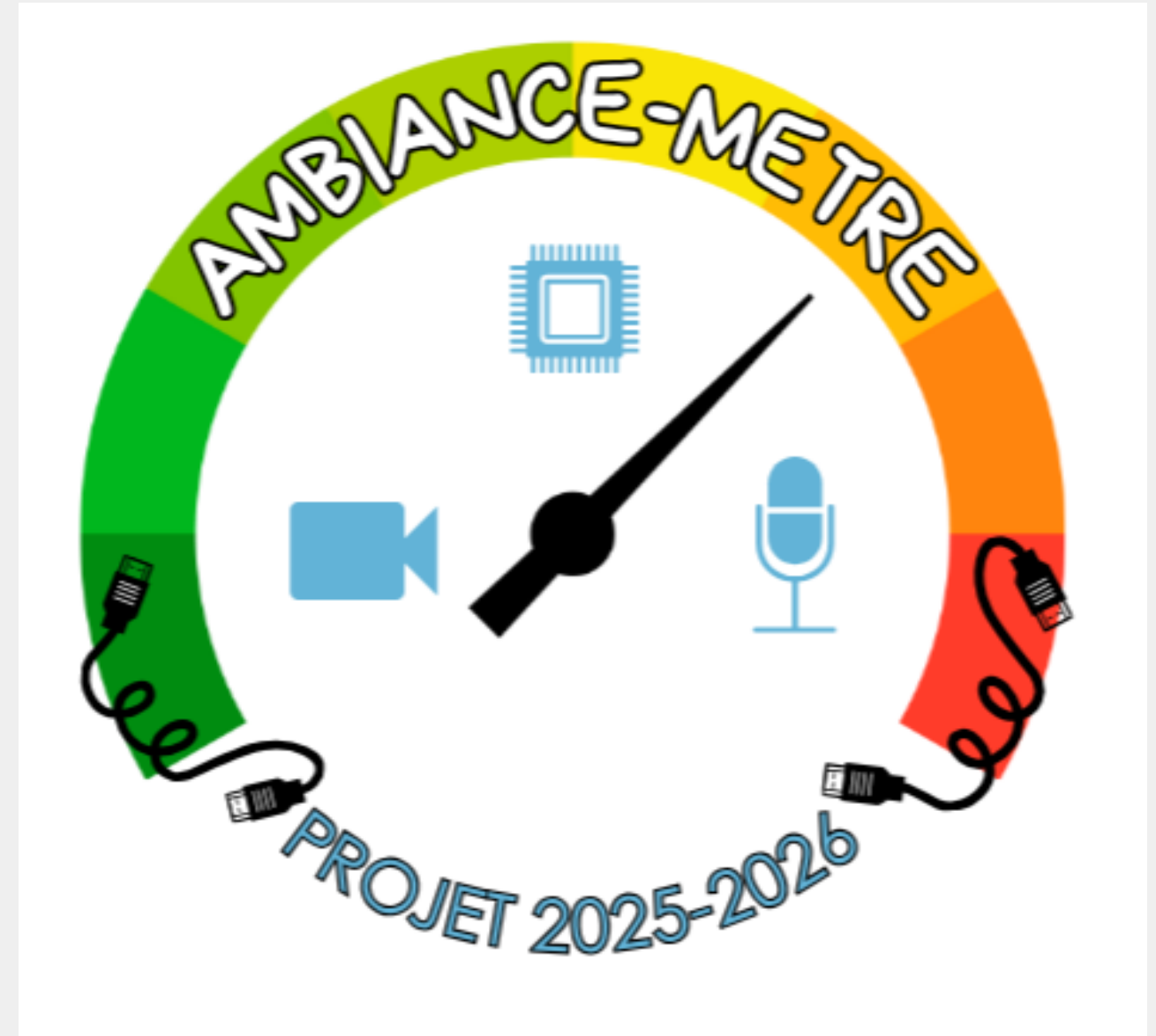
4

Perspectives à venir

5

Conclusion

1) Explication de notre projet



a) Contexte



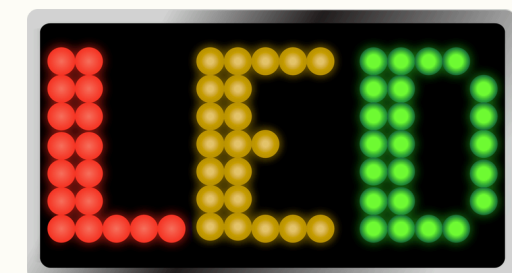
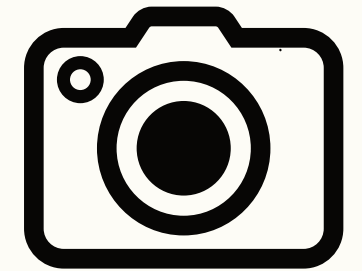
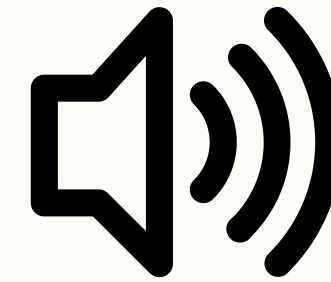
- Patio très fréquenté, mais dont l'activité varie selon les moments de la journée.
- Ambiance est facilement perceptible par les usagers (on la voit, on l'entend), mais elle reste subjective : chacun peut la ressentir différemment.
- Le tuteur (M. Placin) : projet conception d'un détecteur d'ambiance.

-->Comment transformer une perception subjective de l'ambiance d'un lieu en une mesure objective, simple et compréhensible, à partir de données sonores et visuelles ?

b) Objectifs du projet

- Concevoir un dispositif embarqué capable de mesurer l'ambiance d'un espace.
- Utiliser des capteurs simples : microphone, caméra.
- Traduire les mesures en un affichage visuel intuitif via un ruban LED.

À terme, permettre la consultation de l'ambiance en temps réel via un site web.



2) Gestion du projet



1. Notre projet

2. Gestion

3. Etapes

4. Perspectives

5. Conclusion

a) Les membres



Emma Rougeron



Alexandra Ruiz



Clotilde Doucet



Juliette Lermusieaux

- Chaque membre est impliqué à la fois dans la gestion de projet et dans les aspects techniques.
- Flexibilité et vision globale du projet.

b) Outils et organisation interne

- Outils de communication et de travail commun
- Recherches et états de l'art
- Méthode de travail progressive
- Réunions au moins une fois par semaine



1. Notre projet

2. Gestion

3. Etapes

4. Perspectives

5. Conclusion

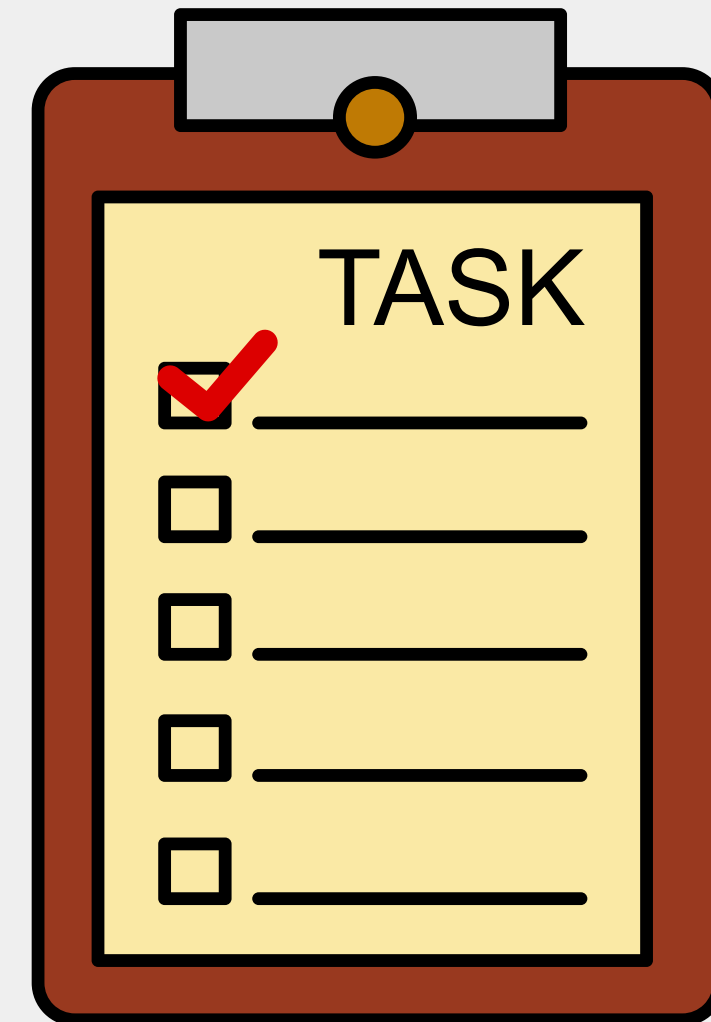
Planning Prévisionnel / Reel

	Septembre	Octobre				Novembre				Décembre				Janvier		
Tâches	26/09	6/10	17/10	22/10	24/10	3/11	7/11	10/11	28/11	5/12	12/12	15/12	17/12	8/1	15/1	19/1
Initialisation																
Définition des besoins																
Cahier des charges																
Matrice d'implication + planning																
Formations																
Développement web																
Maquettage sur Figma																
Programmation micro-contrôleur																
Prototype V1																
Prise en main du matériel																
Tests des différents composants																
Création du prototype V1																
Programmation du matériel																
Calibrage des seuils d'ambiance																
Maquettage																
Réalisation de la maquette fonctionnelle																
Développement																
Développement du site web																
Prototype V2																
Installation du matériel sur le patio																
Tests finaux du prototype																
Ajustements du prototype																
Retour																
Création et envoi du questionnaire																
Analyse des résultats (questionnaires)																
Réajustement du prototype																

c) Identification des risques

Risque	Probabilité (/5)	Gravité (/10)	Prévention	Correction
Mauvaise calibration des seuils sonores	3	9	Tests et ajustement progressif des seuils	Recalibrer les seuils et améliorer le traitement du signal
Difficulté à représenter tout le patio avec un seul micro et une seule caméra	4	8	Choisir un emplacement stratégique pour les capteurs	Déplacer le dispositif ou ajouter des capteurs si nécessaire
Manque de performance du programme en Micropython (latence, réactivité des LED)	3	7	Optimiser le code et limiter les calculs inutiles	Réécrire certaines parties du programme
Difficulté de synchronisation entre le microcontrôleur et le site web	4	8	Réfléchir à l'échange de données dès la conception	Modifier le code et la façon d'envoyer les données
Manque d'intérêt des utilisateurs pour le dispositif	3	5	Rendre l'affichage compréhensible et faire de la communication	Recueillir des retours utilisateurs et améliorer le dispositif
Conditions météo : court-circuit et dégradation du matériel	2	10	Choix d'un emplacement protégé, boîtier de protection	Mise hors tension du système, remplacement du matériel endommagé et déplacement vers un lieu plus sûr

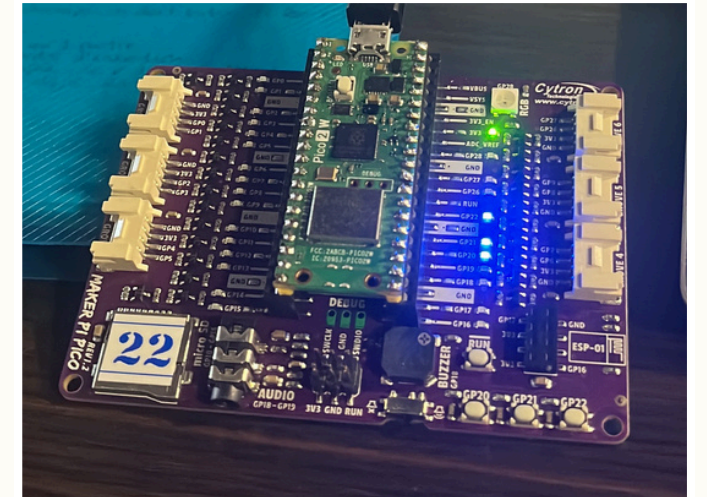
3) Étapes réalisées et résultats



Étape 1 : Prise en main du microcontrôleur et du matériel



Bonne compréhension de son fonctionnement.



```
1 from machine import Pin
2 from time import sleep_ms
3 * led=Pin(3, Pin.OUT)
4
5 led.off()
6
7 while True:
8     led.on()
9     sleep_ms(100)
10    led.off()
11    sleep_ms(100)
```

Tester une broche du microcontrôleur :

- Configurer la broche GP3 comme une sortie pour y brancher une LED.
- La LED est d'abord éteinte, puis :
 1. elle s'allume pendant 100 ms (=0.1s),
 2. elle s'éteint pendant 100 ms
 3. cycle répété en continu.

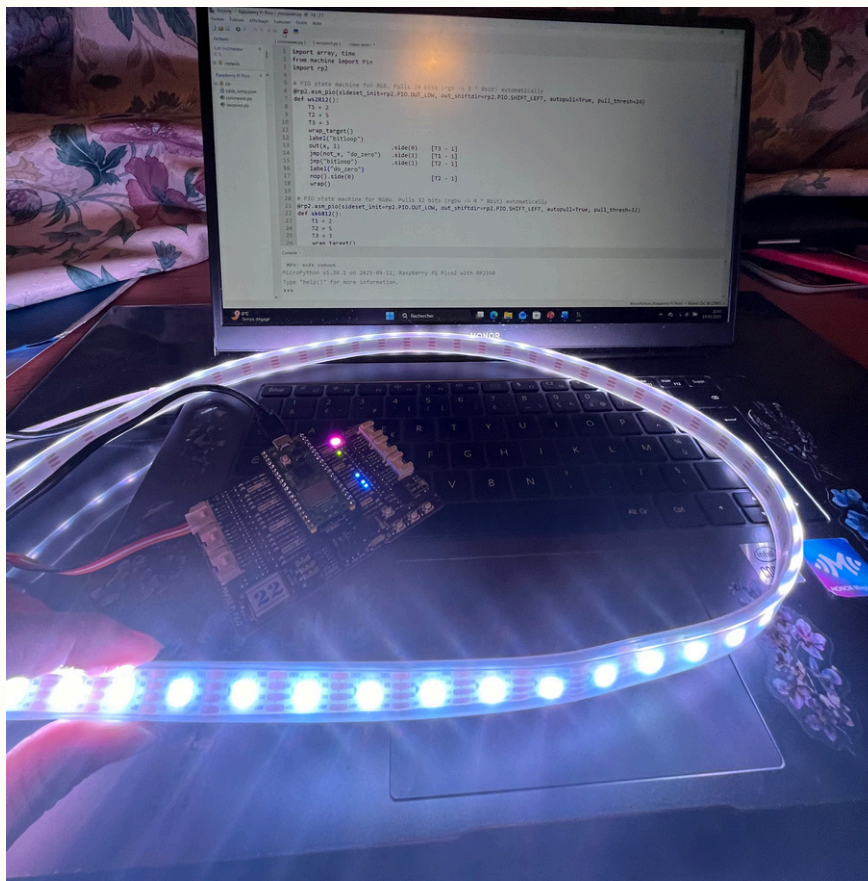
* pour éteindre les LED déjà
allumées :

```
led3=Pin(3, Pin.OUT)
led20=Pin(20, Pin.OUT)
led21=Pin(21, Pin.OUT)
led22=Pin(22, Pin.OUT)

led3.off()
led20.off()
led21.off()
led22.off()
```

Étape 2 : Allumage du ruban LED seul

2.1) Ruban LED en blanc :

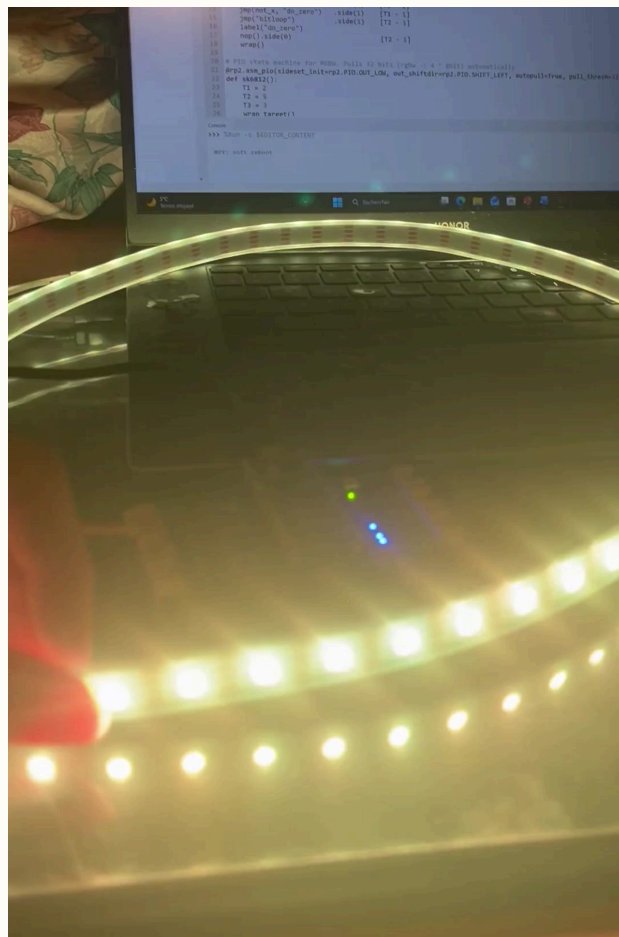


```
1 from neopixel import Neopixel
2
3 numpix = 60
4 pin = 1
5
6 strip = Neopixel(numpix, 0, pin, "GRB")
7
8 strip.fill((255, 255, 255))
9 strip.show()
10
11 while True:
12     pass
```

- Importer la bonne bibliothèque
- Définir nombre de LEDS, la broche de sortie
- Création objet Strip = la bande LED
- 3 couleurs mises à fond = blanc
- strip.show() : envoie la couleur au ruban
- Boucle infinie

Étape 2 : Allumage du ruban LED seul

2.2) Alternance des 3 couleurs :

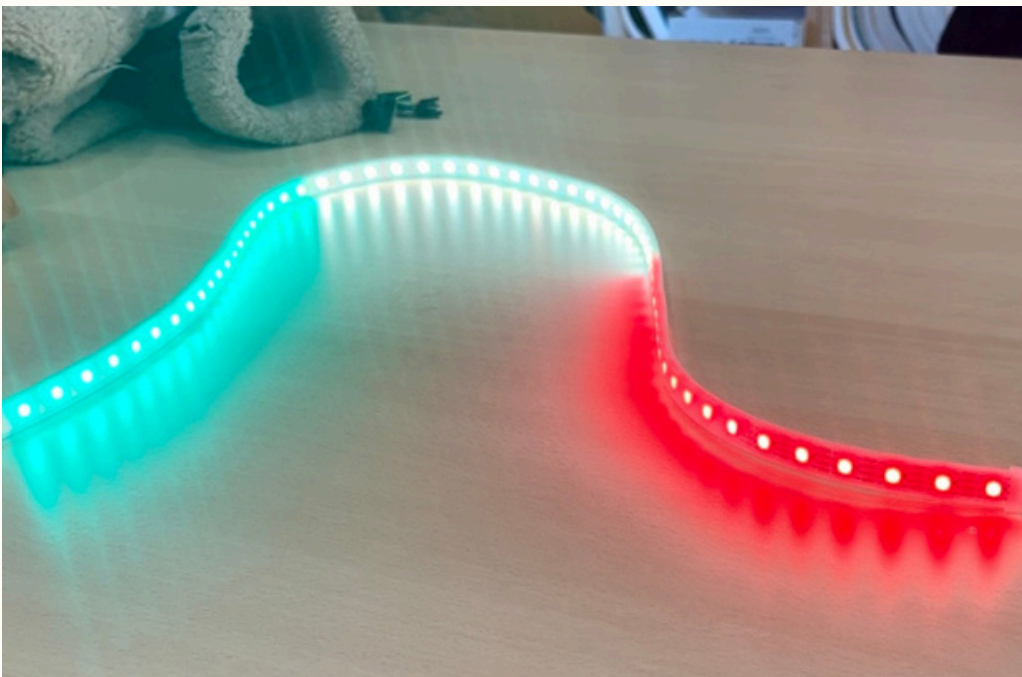


```
1 from neopixel import Neopixel
2 import time
3
4 numpix = 60
5 pin = 1
6
7 strip = Neopixel(numpix, 0, pin, "GRB")
8
9 while True:
10     strip.fill((0, 255, 0)) #vert
11     strip.show()
12     time.sleep(1)
13
14     strip.fill((255, 255, 0)) #jaune
15     strip.show()
16     time.sleep(1)
17
18     strip.fill((255, 0, 0)) #rouge
19     strip.show()
20     time.sleep(1)
```

- `strip.fill((R,G,B))` : prépare toutes les LED avec la couleur choisie
- donner les bonnes valeurs des couleurs
- `time.sleep(1)` : pause d'une seconde par couleur

Étape 2 : Allumage du ruban LED seul

Étape 2.3) Affichage par zones (20 LED par couleur) :



```
1 from neopixel import Neopixel
2
3 numpix = 60
4 pin = 1
5
6 strip = Neopixel(numpix, 0, pin, "GRB")
7
8 #rouge
9 strip.set_pixel_line(0, 19, (255, 0, 0))
10
11 #jaune
12 strip.set_pixel_line(20, 39, (255, 255, 0))
13
14 #vert
15 strip.set_pixel_line(40, 59, (0, 255, 0))
16
17 strip.show()
18
19 while True:
20     pass
```

- `strip.set_pixel_line(start, end, (R,G,B))` : nouvelle fonction qui colore un segment précis de LED.

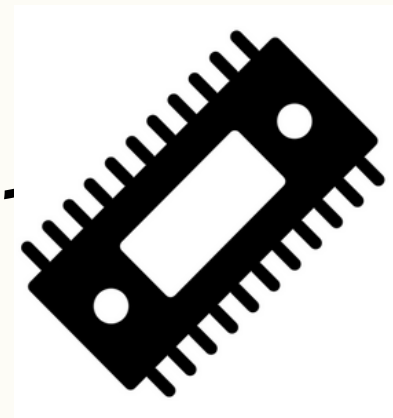
Étape 3.1 : Mesure et traitement du son :

1 **Vérification matériel + Recherches**
=> Lecture **valeurs** brutes du microphone



Microphone

tension



Micro-contrôleur

Conversion

valeur numérique comprise
entre [0; 65535]

bruit ambiant

Code :

```
1 from machine import ADC
2
3 mic = ADC(26)
4
5 while True:
6     print(mic.read_u16())
```

tension mesurée (voltage) en numérique

➔ Transformer cette valeur numérique en donnée exploitable

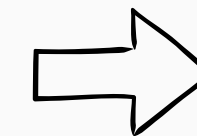
Étape 3.2 : Mesure et traitement du son

2 Conversion
tension mesurée en **numérique** → tension en **volt**

```
1 from machine import ADC
2
3 mic = ADC(26)
4
5 while True:
6     U = (mic.read_u16() / 65535) * 3.3
7     print(U)
```

3 a) Approximation de niveau sonore en décibels.
b) Création du code micro-python qui convertit la tension (V) en niveau sonore (dB)

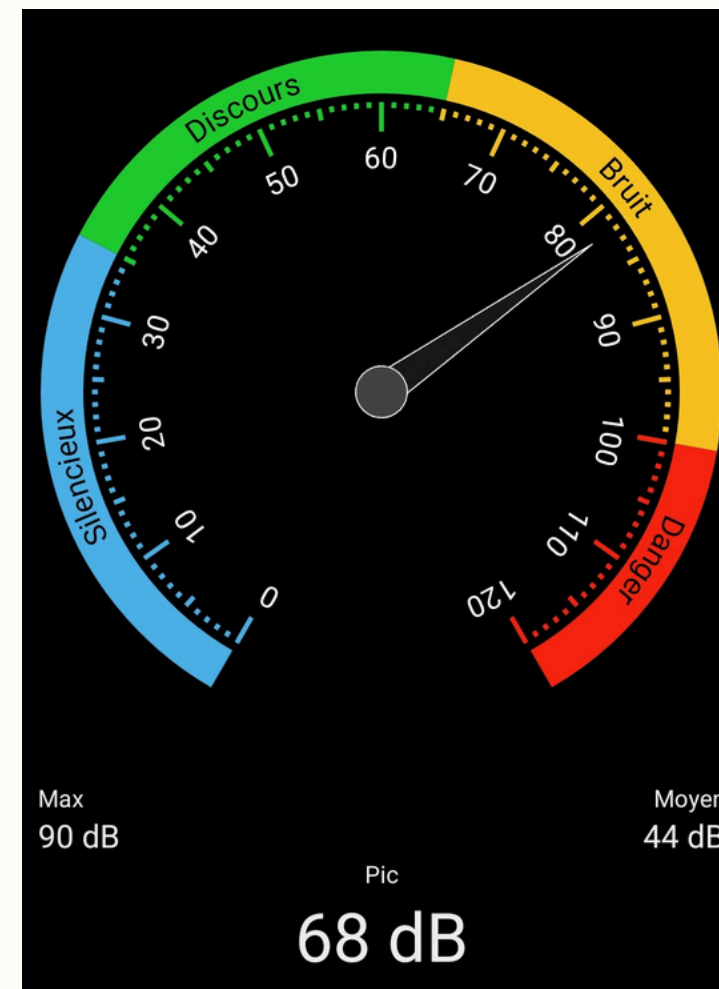
```
1 from machine import ADC
2 mic= ADC(26)
3
4 while True:
5     volt = (mic.read_u16() / 65535) * 3.3
6
7     if volt >= 0.6:
8         dB = volt* 50
9     else:
10        dB = 0
11
12 print("Niveau sonore :", round(dB,1), "dB")
```



Création d'une boucle if pour renvoyer les tensions au-dessus du seuil de référence 0.6

Étape 3.3 : Mesure et traitement du son : Tests

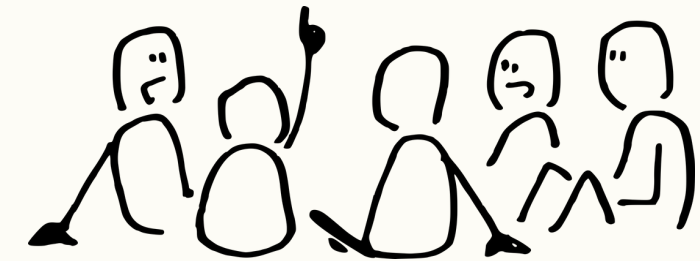
- 4 Multiples tests en conditions réelles :
- Utilisation application qui mesure le nombre de décibel
 - Comparaison avec le nombre de dB renvoyé par le programme



Cris



Discussions



Silence 



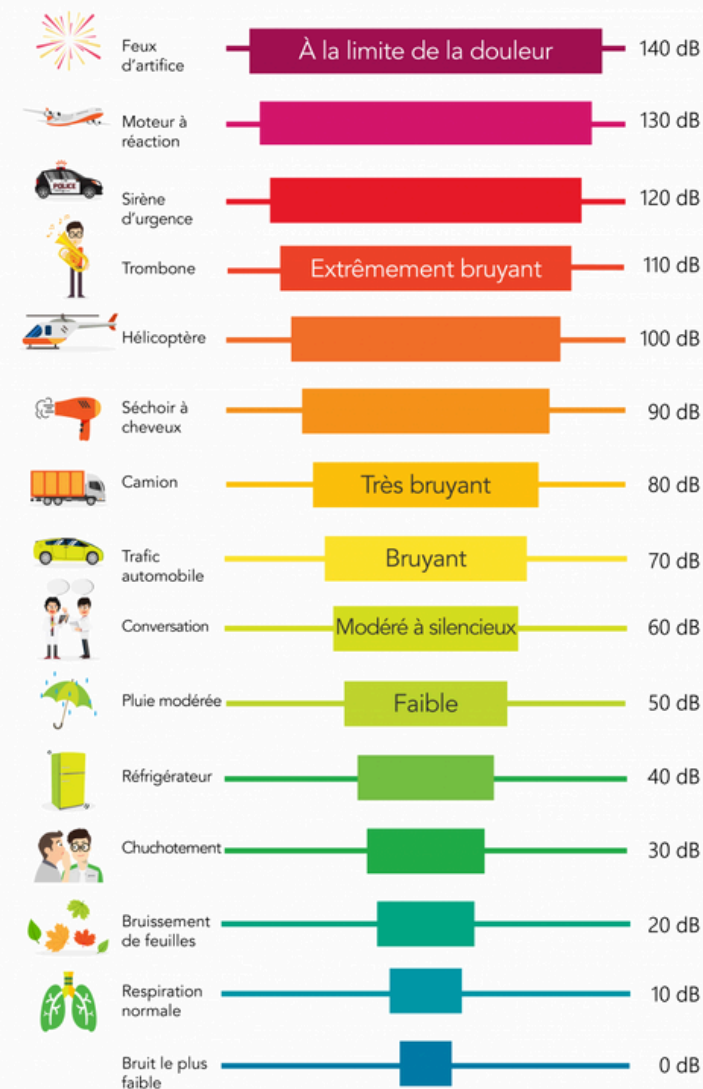
Étape 4.1 : Couplage son + LED (prototype fonctionnel)



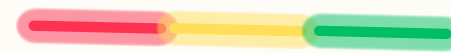
Relier le niveau sonore à l'affichage LED

1. Définir les seuils d'ambiance (dB)

Échelle des décibels (dB)



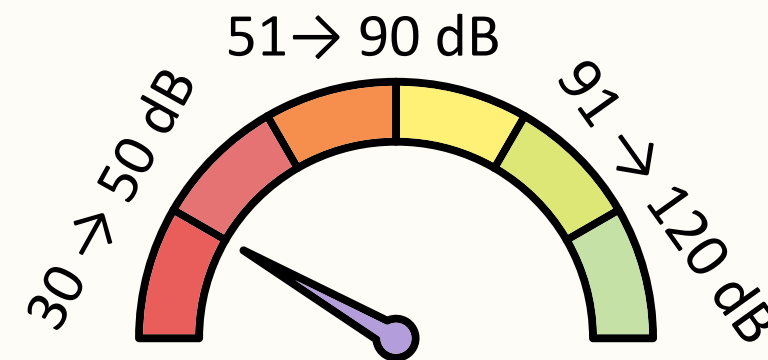
2. Choix des couleurs du ruban LED selon les seuils d'ambiance



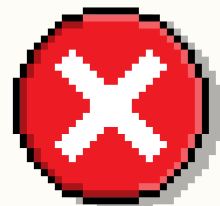
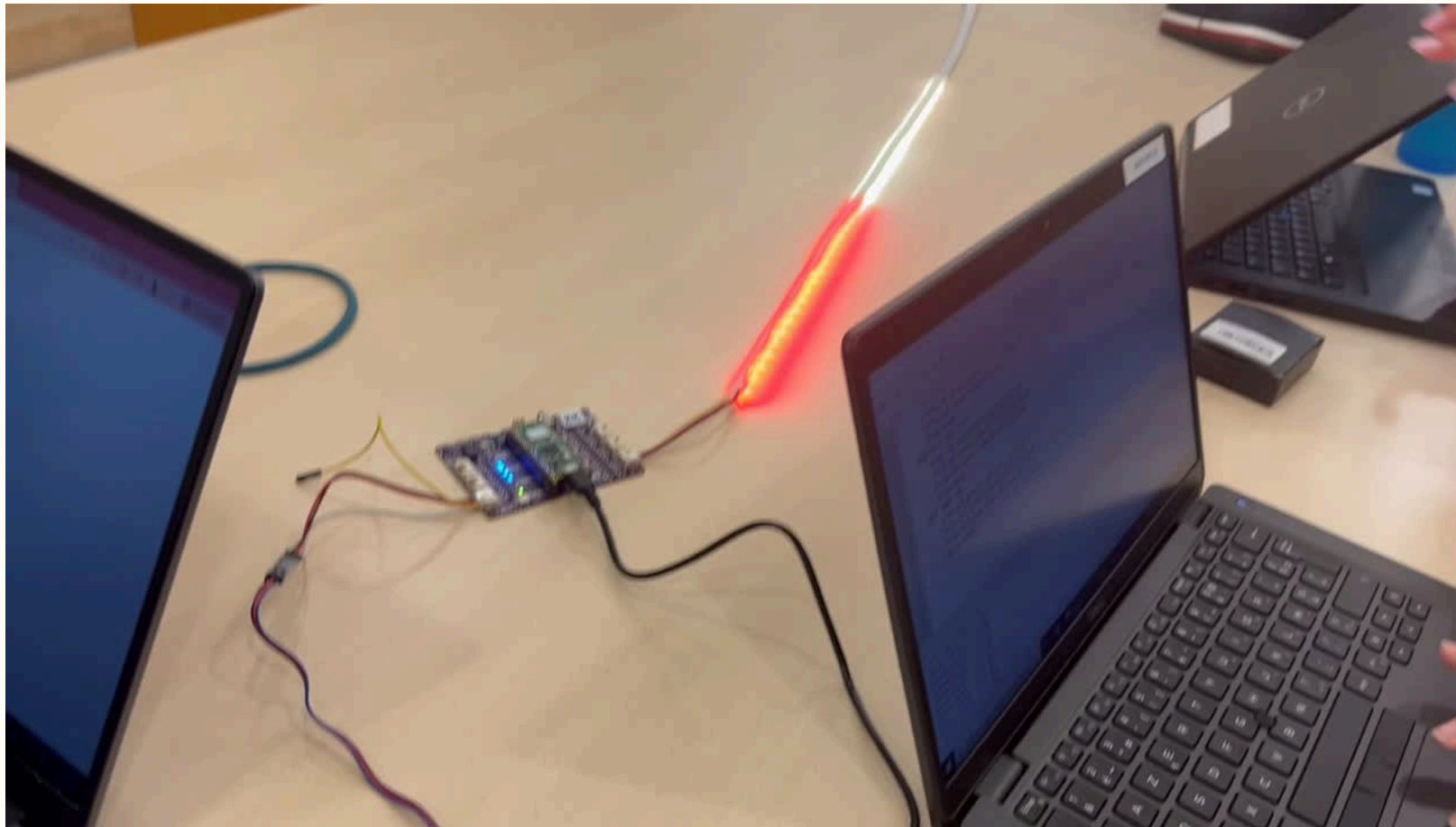
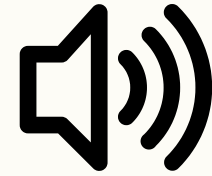
20
LEDS

Rouge : ambiance calme
Jaune : ambiance modérée
Vert : ambiance animée

Allumage progressif du ruban LED 5 LEDS tous les 10 dB



Étape 4.2 : Résultats



Problèmes :

- Code long
- Beaucoup de elif
- Allumage de LEDS 5 par 5

```

1 from machine import ADC
2 import time
3 from neopixel import Neopixel
4
5 numpix = 60
6 pin = 1
7
8 strip = Neopixel(numpix, 0, pin, "GRB")
9 mic = ADC(26)
10
11 while True:
12     volt = (mic.read_u16() / 65535) * 3.3
13     if volt >= 0.6:
14         dB = volt * 50
15     else:
16         dB = 0
17     print("Niveau sonore :", round(dB, 1), "dB")
18     time.sleep(0.1)
19     strip.fill((0,0,0))
20     if 30 < dB <= 35:
21         strip.set_pixel_line(0, 4, (255,0,0))      # 5 LEDS
22     elif 35 < dB <= 40:
23         strip.set_pixel_line(0, 9, (255,0,0))      # 10 LEDS
24     elif 40 < dB < 50:
25         strip.set_pixel_line(0, 19, (255,0,0))     # 20 LEDS max rouge
26     elif 51 < dB <= 60:
27         strip.set_pixel_line(0, 19, (255,0,0))
28         strip.set_pixel_line(20, 24, (255,255,0))  # 5 LEDS
29     elif 60 < dB <= 70:
30         strip.set_pixel_line(0, 19, (255,0,0))
31         strip.set_pixel_line(20, 29, (255,255,0))  # 10 LEDS
32     elif 70 < dB <= 80:
33         strip.set_pixel_line(0, 19, (255,0,0))
34         strip.set_pixel_line(20, 34, (255,255,0))  # 15 LEDS
35     elif 80 < dB <= 90:
36         strip.set_pixel_line(0, 19, (255,0,0))
37         strip.set_pixel_line(20, 39, (255,255,0))  # 20 LEDS max jaune
38     elif 91 < dB <= 100:
39         strip.set_pixel_line(0, 19, (255,0,0))
40         strip.set_pixel_line(20, 39, (255,255,0))
41         strip.set_pixel_line(40, 44, (0,255,0))   # 5 LEDS
42     elif 101 < dB <= 110:
43         strip.set_pixel_line(0, 19, (255,0,0))
44         strip.set_pixel_line(20, 39, (255,255,0))
45         strip.set_pixel_line(40, 49, (0,255,0))   # 10 LEDS
46     elif 110 < dB <= 120:
47         strip.set_pixel_line(0, 19, (255,0,0))
48         strip.set_pixel_line(20, 39, (255,255,0))
49         strip.set_pixel_line(40, 54, (0,255,0))   # 15 LEDS
50     elif dB > 120:
51         strip.set_pixel_line(0, 19, (255,0,0))
52         strip.set_pixel_line(20, 39, (255,255,0))
53         strip.set_pixel_line(40, 59, (0,255,0))   # 20 LEDS max vert
54 strip.show()

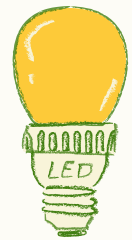
```

Étape 5.1 : Affichage progressif



Créer une boucle for qui permette d'allumer une LED par une LED

1. Recherche du nombre de LEDS à allumer

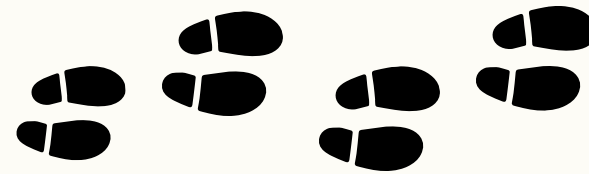


Au départ :

- Allumer une LED tous les 5 dB

Après calculs :

- 5 dB → Intervalle trop grand
- Allumer une LED tous les 1,5 dB

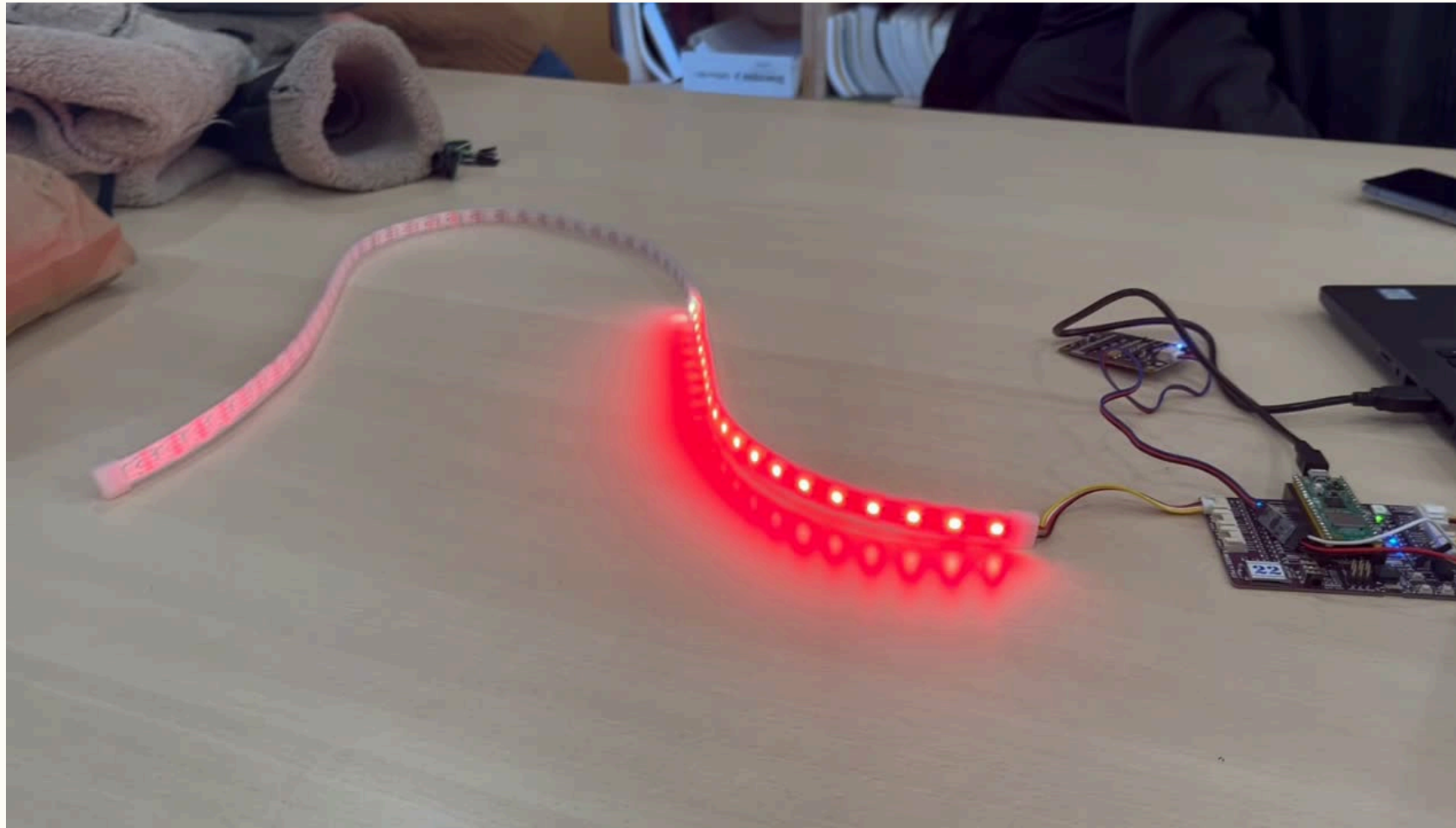


2. Calcul permettant de savoir combien de LED il faut afficher

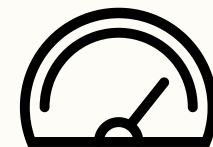
$$\text{nb de LED à afficher} = \frac{(\text{dB obtenus} - \text{dB minimal})}{1,5}$$

dB minimal= 40 dB

Étape 5.2 : Résultats



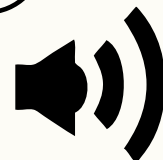
✓ Prototype fonctionnel réagissant



en temps réel



au bruit ambiant



```

1 from machine import ADC
2 import time
3 from neopixel import Neopixel
4
5 numpix = 60
6 pin = 1
7
8 strip = Neopixel(numpix, 0, pin, "GRB")
9 mic = ADC(26)
10
11 dB_min = 40          # dB où la première LED rouge s'allume
12 dB_pas = 1.5         # 1 LED tous les 1.5 dB
13
14 while True:
15     volt = (mic.read_u16() / 65535) * 3.3
16
17     if volt >= 0.6:
18         dB = volt * 50
19     else:
20         dB = 0
21
22     print("Niveau sonore :", round(dB, 1), "dB")
23
24     strip.fill((0, 0, 0))
25
26     leds = int((dB - dB_min) / dB_pas)
27
28     if dB < 0 :
29         leds = 0
30
31     time.sleep(0.1)
32     strip.fill((0,0,0))
33     for i in range(leds):
34         if 40 < dB < 70:
35             strip.set_pixel_line(0, i, (255, 0, 0)) # Rouge
36         elif 71 < dB < 100:
37             strip.set_pixel_line(0, 19, (255, 0, 0)) # début rouge
38             strip.set_pixel_line(19, i, (255, 255, 0)) # jaune
39         elif 101 < dB < 130:
40             strip.set_pixel_line(0, 19, (255, 0, 0)) # début rouge
41             strip.set_pixel_line(19, 39, (255, 255, 0)) # début jaune
42             strip.set_pixel_line(39, i, (0, 255, 0)) # Vert
43     strip.show()
44 
```


4) Perspectives et suite du projet



Étapes à venir

Caméra et indicateur d'ambiance



- Caméra ArduCAM OV2640 → intégrer pour détecter présence et mouvements
- Travail de recherche mené → il faut finaliser le code et effectuer les branchements
- Fusion audio/vidéo → développer l'indicateur d'ambiance plus fiable

Site web

Exploiter et afficher l'ambiance en temps réel, valoriser les données



Tests utilisateurs

Vérifier LED et cohérence dispositif/site avant finalisation

1. Notre projet

2. Gestion

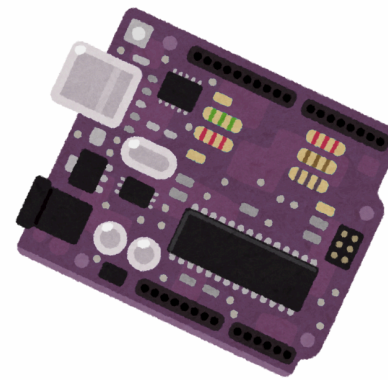
3. Etapes

4. Perspectives

5. Conclusion

Compétences développées

Programmation embarquée (MicroPython / microcontrôleur)



Gestion de projet souple



Pilotage de capteurs et modules matériels



Travail d'équipe et coordination

Développement web (à venir)

RUIZ Alexandra, ROUGERON Emma, LERMUSIEAUX Juliette, DOUCET Clotilde

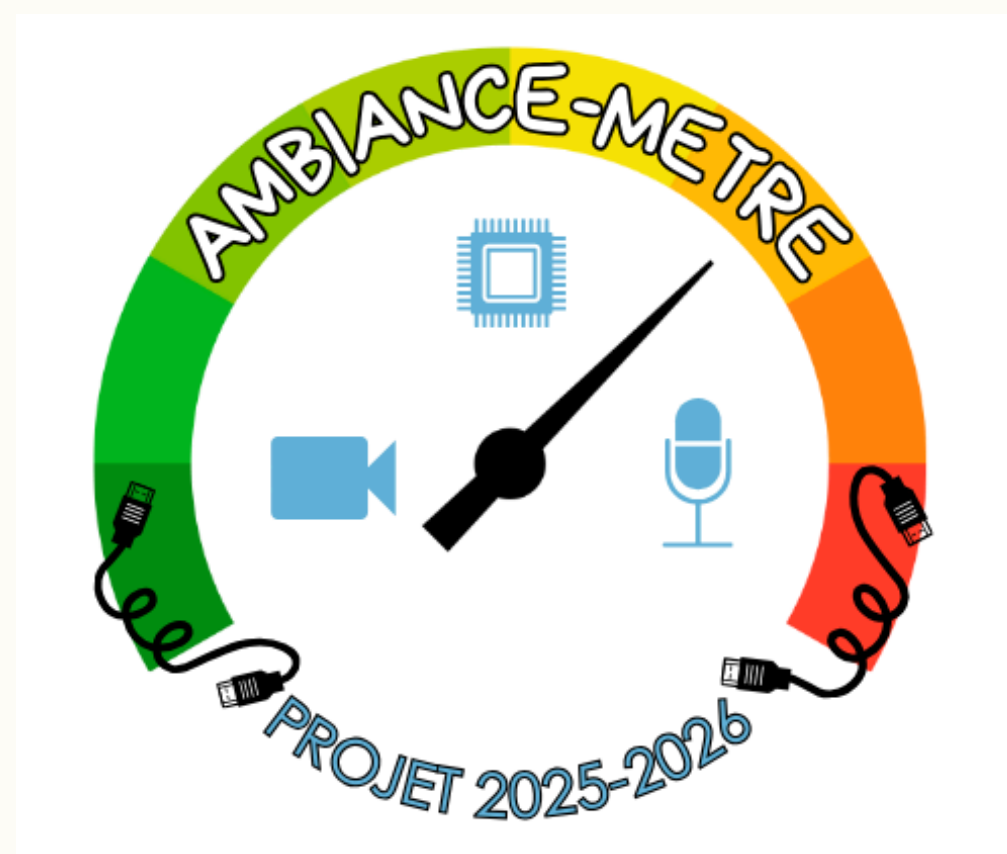
Conclusion

- ✓ Premier prototype fonctionnel validé
- ✓ Organisation de projet cohérente et structurée

Prochaines étapes :

- Intégration de la vision
- Affiner les seuils d'analyse de l'ambiance
- Visualisation en temps réel (site web)

Merci !



Pour votre attention.